

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

Thesis title

Livia Cavalcanti Bandeira Julia o

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

Thesis title

Titel der Abschlussarbeit

Author:	Livia Cavalcanti Bandeira Julio o
Supervisor:	Prof. Dr. Jana Giceva Makreshanska
Advisor:	Fabian Wenz
Submission Date:	Submission date

I confirm that this master's thesis in data engineering and analytics is my own work and I have documented all sources and material used.

Munich, Submission date
lia o

Lívia Cavalcanti Bandeira Ju-

Acknowledgments

Abstract

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
1.1 Section	2
1.2 Research Background	2
1.2.1 Problem Statement	2
1.2.2 Research Objectives	2
2 Literature Review	4
2.1 Literature Review	4
3 Dataset	5
3.1 Dataset	5
3.1.1 AutoPatch	5
3.1.2 CVEfixes	5
3.2 CVEfixes Dataset Integration	5
3.2.1 CVEfixes Dataset Integration: Overview	5
3.2.2 CVEfixes: CWE Distribution & Filtering	7
4 Methodology	10
5 Results	20
5.0.1 File vs. Method	20
List of Figures	23
List of Tables	24
Bibliography	25

1 Introduction

Identifying where the vulnerability in previous fixes consists of, given a code change that address the vulnerability, the removed code and/or the added code are the fixes and the code around it are where the vulnerability resided. This code changes are done by developers and logged in systems that catalog the vulnerabilities. Once a vulnerability is identified in one project, it can be easily exploited in other projects. therefore, other projects, especially those open-source, might quickly address the issue. this require verifying continuously the catalog of vulnerabilities which is a toll for the software development process and usually require specialized professionals on companies. This task on open source projects where many contributions are made by voluntary developers drag the development speed, at the sama time as not addressing them may hurt the adoption of the developed code by the users.

Once a vulnerability is identified and fixed, we have an expansion of the commu-natary knowledge. and fixing it in other projects will roughly consists of applying a similar change in a similar context as the first one identified. This workflow works specially well when we have open catalogs of the vulnerabilities that were addressed. This shared knowledge can also be useful for enterprises, that basically have to go through the same process. But the developpers are dedicated to a set of tasks and among them they might deal with the same security monitoring toll as the open source groups.

The usage of an automated tool does not aim to vanish the need of such security activities. As the proposed pipeline only works if a developer fixed a vulnerability in the first step. Instead, it'S goal is to maximize the propagation of the human knowledge, free the security experts time to look for other unknown vulnerabilities, reduce the attack window for known vulnerabilities, and allow that even teams without securities experts can have their code free from vulnerabilities.

1.1 Section

1.2 Research Background

To meet compliance requirements, organizations must adhere to International Standards and regulations such as ISO/IEC 27001 International Organization for Standardization, 2022. This standard establishes best practices for data protection, cyber-resilience, and operational excellence. To accomplish that and improve development robustness, enterprises must anticipate and adapt to vulnerabilities and potential attacks. For software companies, this goal demands evolving software development life cycles to rapidly identify, generate, and deploy risk mitigation for vulnerabilities. One possible approach is to integrate automated systems that proactively generate patches for buggy programs.

With the advent and widespread adoption of Large Language Models (LLMs), a new venue for automated remediation emerged Xia and Zhang, 2024. However, the current LLM approaches remain insufficient for security patching due to two primary failure reasons:

1.2.1 Problem Statement

These failures are aggravated by providing lengthy raw source code to the LLM. In these cases, vulnerability-relevant logic is diluted by implementation details and boilerplate code, thereby degrading the quality of security analysis Yang, Xu, Gao, and Sun, 2025. Empirical results from the same work suggest that this code noise induces what the authors term Analysis Focus Drift, wherein the model performs a code quality review rather than a vulnerability analysis. Thus, there is an opportunity to develop methodologies that enrich security-relevant information while filtering out implementation noise. Moreover, although various studies show that knowledge bases improve LLM responses in security applications Du, Zheng, Wang, et al., 2025; Guo, Nie, Xie, et al., 2025; Seo, Choi, Shin, and You, n.d.; Wang, Jiang, et al., 2025, it is unclear which approach optimizes patch quality effectively in multi-agent environments.

1.2.2 Research Objectives

To evaluate the proposed framework, we address two primary research questions:

RQ1: How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?

RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

If within reach, we aim to also address the following complementary research questions:

CRQ1: How does the structure-aware RAG framework with a small model compare with existing approaches (e.g., zero-shot small models and AutoPatch)?

CRQ2: What is the computational and financial cost of the approach (e.g., token usage, storage complexity and size, end-to-end pipeline costs)?

2 Literature Review

2.1 Literature Review

Structural Retrieval and Graph Edit Distance

Graph Embedding Techniques

Agentic Patching Frameworks

3 Dataset

3.1 Dataset

3.1.1 AutoPatch

The AutoPatch dataset contains high-severity CVEs disclosed in 2024 and 2025 and collected from the GitHub Advisory Database, Openwall [36], and the Chromium issue tracker with a custom crawler. It includes 57 vulnerabilities from the Linux Kernel and 10 from the Chromium project. For each CVE, 75 vulnerable and its respective patched code were generated by using 5 LLMs: Code Llama (13b-instruct), DeepSeek Coder (v2-lite), DeepSeek-R1 (32b), GPT-4o, and OpenAI o3-mini. The augmentation strategies included inserting unreachable code fragments, adding random codes in comments, varying whitespace and formatting, defining a useless function, and introducing extraneous newline characters. (copied and paste from the paper)

3.1.2 CVEfixes

3.2 CVEfixes Dataset Integration

3.2.1 CVEfixes Dataset Integration: Overview

Goal

Integrate the CVEfixes database into our pipeline to obtain a large-scale knowledge base of historical vulnerability fixes. Verify that our CPG+embedding approach generalises beyond the 70-sample AutoPatch corpus.

Database Scale

CVEs 11,873 vulnerabilities

commits 12,107 fix commits

files 51,342 changed files

methods 277,948 method changes

Table 3.1: Distribution of CWE Types in the AutoPatch Dataset ($N = 75$)

CWE ID	Category Group	CWE Name / Description	Count	Ratio (%)
CWE-20	Validation & Logic	Improper Input Validation	20	26.67%
CWE-362	Concurrency	Race Condition / Synchronization	16	21.33%
CWE-252	Validation & Logic	Unchecked Return Value	12	16.00%
CWE-843	Type Errors	Type Confusion	8	10.67%
CWE-416	Memory Safety	Use After Free	6	8.00%
CWE-476	Memory Safety	NULL Pointer Dereference	4	5.33%
CWE-703	Validation & Logic	Improper Handling of Exceptional Conditions	3	4.00%
CWE-122	Memory Safety	Heap-based Buffer Overflow	1	1.33%
CWE-125	Memory Safety	Out-of-bounds Read	1	1.33%
CWE-787	Memory Safety	Out-of-bounds Write	1	1.33%
CWE-400	Resource Handling	Uncontrolled Resource Consumption	1	1.33%
CWE-754	Validation & Logic	Improper Check for Unusual Conditions	1	1.33%
CWE-775	Resource Handling	Missing Release of Resource	1	1.33%
Total			75	100.00%

Predominantly C code (92%) — consistent with kernel/system-level focus of our pipeline. Average patch adds 22 lines and removes 5 lines (asymmetric fixes).

Extracted Corpus Statistics

Metric	Value
Method pairs	4,607
Unique CVEs	1,777
Unique CWEs	92
Languages	C (92%), C++ (8%)
Avg lines added	22.4
Avg lines removed	5.4
Avg similarity ratio	0.563
Severity: CRITICAL	557
Severity: HIGH	1,132
Severity: MEDIUM	1,388

3.2.2 CVEfixes: CWE Distribution & Filtering

The sample contains C and C++ code snippets. It has 328 C entries (72.9%) and 122 C++ entries (27.1%).

Top CWEs in CVEFixes

CWE	Name	Count
CWE-119	Buffer Overflow	507
CWE-125	Out-of-bounds Read	393
CWE-20	Improper Validation	308
CWE-416	Use After Free	304
CWE-476	NULL Ptr Deref	259
CWE-362	Race Condition	250
CWE-787	Out-of-bounds Write	150
CWE-190	Integer Overflow	137
CWE-400	Resource Exhaustion	137
CWE-401	Memory Leak	77

AutoPatch CWE Overlap Filter

Filtered CVEfixes to CWEs present in the AutoPatch dataset (23 CWE types) to ensure the expanded corpus covers the same vulnerability families.

Metric	Value
AutoPatch CWE types	23
CVEfixes entries matching	1,618
Coverage of extracted corpus	35.1%
CVE-based filter (exact CVEs)	0

Zero CVE overlap — purely temporal: CVEfixes ends July 2024; AutoPatch CVEs are from late 2024–2025. The datasets are fully complementary (same CWE families, disjoint time periods).

Table 3.2: CWE Frequency Distribution Across Index and Query (Test) Datasets

Split	CWE / Category	Count (N)	Frequency (%)	Unique Entities
Index	CWE-362	60	16.7%	Unique CVEs: 26 Unique CWEs: 6
	CWE-476	60	16.7%	
	CWE-416	60	16.7%	
	CWE-20	60	16.7%	
	CWE-190	60	16.7%	
	CWE-787	60	16.7%	
	<i>Subtotal</i>	<i>360</i>	<i>100.0%</i>	
Query (Test)	CWE-362	15	16.7%	Unique CVEs: 25 Unique CWEs: 6
	CWE-476	15	16.7%	
	CWE-416	15	16.7%	
	CWE-20	15	16.7%	
	CWE-190	15	16.7%	
	CWE-787	15	16.7%	
	<i>Subtotal</i>	<i>90</i>	<i>100.0%</i>	
Total (Combined)	CWE-362	75	16.7%	Total CVEs: 27 Total CWEs: 6
	CWE-476	75	16.7%	
	CWE-416	75	16.7%	
	CWE-20	75	16.7%	
	CWE-190	75	16.7%	
	CWE-787	75	16.7%	
	Dataset Total	450	100.0%	

Table 3.3: Frequency of Top CVEs Across Index, Query, and Combined Datasets

CVE ID	Index ($n = 360$)		Query ($n = 90$)		Combined ($n = 450$)	
	Count	Freq (%)	Count	Freq (%)	Count	Freq (%)
CVE-2021-26788	60	16.7%	15	16.7%	75	16.7%
CVE-2023-36326	60	16.7%	15	16.7%	75	16.7%
CVE-2020-15211	60	16.7%	15	16.7%	75	16.7%
CVE-2012-3552	24	6.7%	8	8.9%	32	7.1%
CVE-2016-3841	14	3.9%	3	3.3%	17	3.8%
CVE-2020-12652	14	3.9%	2	2.2%	16	3.6%
CVE-2019-18885	13	3.6%	1	1.1%	14	3.1%
CVE-2020-10739	11	3.1%	3	3.3%	14	3.1%
CVE-2019-11487	8	2.2%	5	5.6%	13	2.9%
CVE-2020-35538	9	2.5%	3	3.3%	12	2.7%
Total Records	360	100.0%	90	100.0%	450	100.0%

4 Methodology

The goal here is to use historical knowledge from previous fixes to fix other code with the same vulnerability.

Architecture

This project presents a structure-aware retrieval system that identifies the type of vulnerability and suggests a fix patch given a code snippet.

Data Pipeline

The process consists of, first, building the index, an offline process:

- loading a vulnerable code snippet
- building a CPG from the code
- embedding the graph
- indexing the embeddings into a vector database, FAISS

then the online process consists of:

- transforming the code snippet input into an embedding
- querying the vector database to find similar code, indicating they contain a similar vulnerability
- using the index data, especially the patched code to prompt an LLM to fix the vulnerability.

On development phase, it is also possible to evaluate retrieval and patch quality.

Graph Generation

Graph Loading

This work utilizes Joern to generate the Code Property Graph (CPG) from a code snippet. For snippets that are not well-formed C/C++ programs, notably those from AutoPatch, the pipeline first wraps the code with the minimum necessary macros and type definitions (e.g. `typedef unsigned int u32, #define NULL ((void*)0)`) and writes a parseable source file. The C/C++ code files are exported to a binary graph with the Joern shell command `joern-parse --language newc`, where `newc` specifies the C++ specificities avoiding will-formed graphs. Then, the compiled graph is exported to GraphML format with the shell-command `joern-export --repr cpg --format graphml`. Each export generates a directory tree where each subdirectory contains an `export.xml` file with a subgraph of the the whole CPG. The nodes can have different labels (e.g. `CALL`, `IDENTIFIER`, `CONTROL_STRUCTURE`), whereas the edges can belong to one of the following graph representation: Abstract Syntax Tree (AST), Control Flow Graph (CFG), Call Dependency Graph (CDG), and Program Dependency Graph (PDG).

The pipeline generates both *vulnerable* and *patched* code snippet graphs.

Graph Loading

The exported graph is imported into the pipeline using the NetworkX library as a `MultiDiGraph`. All the `export.xml` discovered by recursively exploring the subdirectories. Once the graph they store is imported to the pipeline, it is merged by reading each file with `nx.read_graphml` and updating the combined graph iteratively. The NetworkX library considers each graph, node and edges as a dictionary and update the data structure by adding one graph set of nodes and edges at a time. The Joern definition of the graph is, that is, the node and edge attributes, is conserved, therefore, the graph consistency depends on the Joern export. Once the graph is loaded, the following cleanups are applied:

- Nodes with the label `COMMENT` or `UNKNOWN` are removed
- Any remaining edges whose source or target were removed are pruned.

Graph Sampling

Once the CPG is loaded, it contains the full code, including the boilerplate added in the pipeline. Therefore it is necessary to isolate the subgraph related to the vulnerability from the irrelevant rest of the code. For that, a subsample is calculated from the difference between the vulnerable graph and the patched graph, the so called G_{vuln} . By doing

so we expect to be able to generalize the search for the vulnerability removing the specificities of a given system that are inprint in the graph.

Bounded slicing To include context that may be relevant, a bounded search is performed with the goal of include any relevant settings in which the vulnerability occurs. The slice is a configurable neighbourhood expansion ($n \in \{0, 1, 2, 3\}$ hops). $n = 0$ is the original sampled graph, that is, only directly changed nodes.

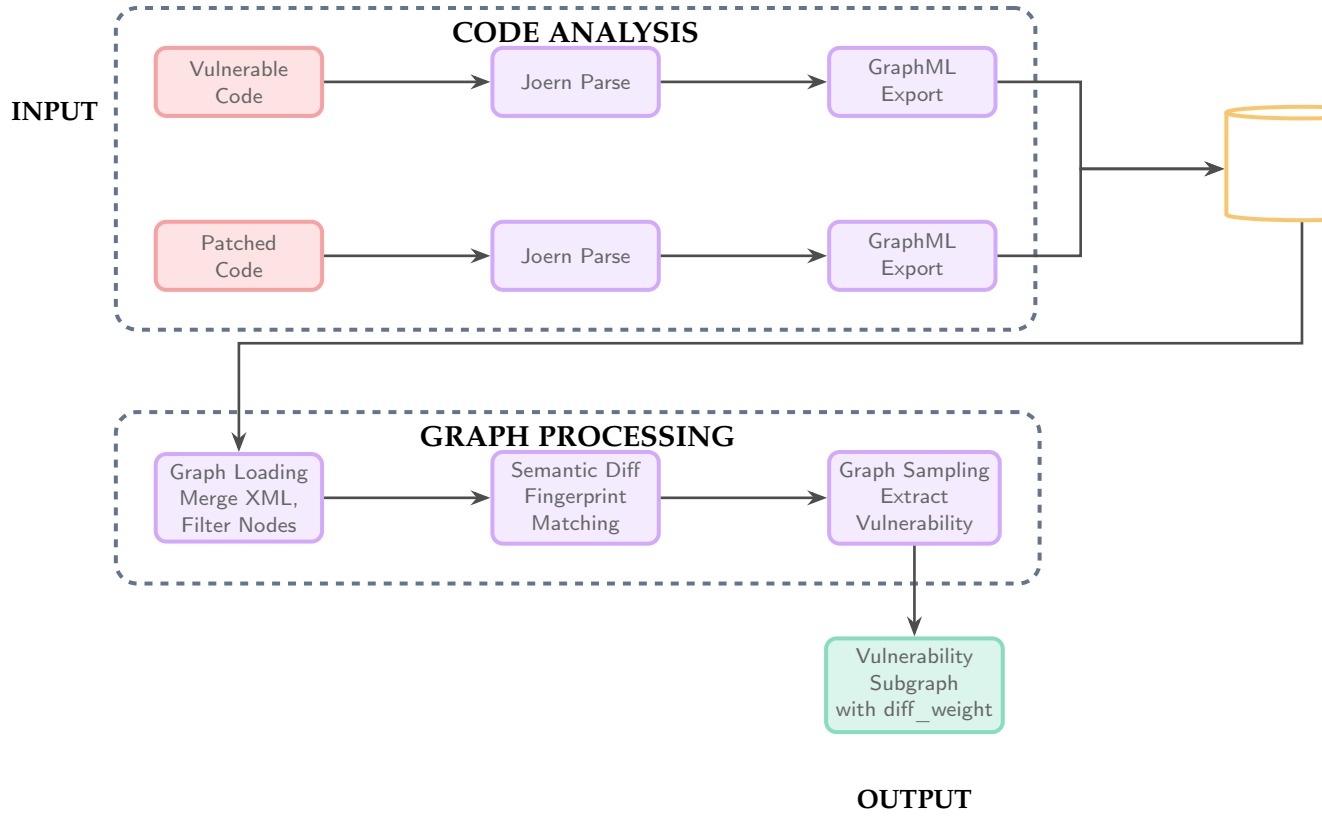
Semantic difference Before perform the sampling, each node needs to be deterministically identifiable to avoid different results depending on runs or randomness resulting from non-deterministic ID assigned by Joern across runs. The nodes in the sample are further enriched with semantic labels resulting of the two graphs comparison. Each node of the sample is labeled as either:

- Removed node: every node present in G_{orig} but absent in the G_{patch} .
- Fix-adjacent node: every node present in G_{orig} that is a removed node neighbor.
- Edge-changed node: every node attached to an edge which any of the nodes changed between the two graphs.

Besides the labelling, the G_{vuln} nodes also received weights.

Label	Weight
removed	1.0
fix_adjacent	0.8
edge_changed	0.6
context	0.2

These weights are directly proportional to the relation of the node to the vulnerability and will be integrated to the embedders for further pruning.



Semantic difference

Embedders

For reference, we started with deterministic embedders that work either with heuristics or with well-defined mathematical formulations. All embedders are **unsupervised**, that is, no labelled training data required. By default their output is L2-normalised vectors with a default dimension of 128, but configurable. Besides that, the embedding may have a PCA.

- **NetLSD** (*Network Laplacian Spectral Descriptor*) Tsitsulin, Mottin, Karras, et al., 2018. Computes the heat trace of the normalised graph Laplacian across a timescale that aims to get graph expressive representations similar to random walk models. It is node permutation-invariant and captures global spectral structure, adapting to scale encoding both local and global features, that is, it is scale-adaptable. It is also capable of comparing different size graphs. It supports weighted graphs. It was created to be an alternative to the NP-approach graph

edit distance, being then efficiently computable, so the comparison calculation happens in a constant time.

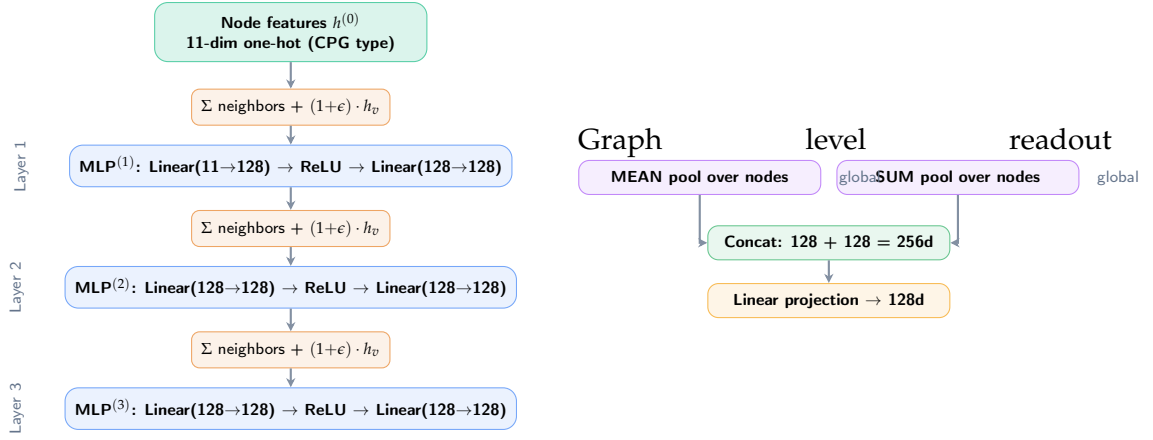
- **WL** (*Weisfeiler-Lehman Kernel*). Iterative colour refinement over node types followed by weighted sum pooling and a linear projection. In this work, it was implemented in PyTorch Geometric with no external dependency. WL test iteratively : (1) aggregates the labels of nodes and their neighborhoods, and (2) hashes the aggregated labels into unique new labels
- **GIN** (*Graph Isomorphism Network*) Xu, Hu, Leskovec, and Jegelka, 2019 Message-passing neural Network that is as powerful as the Weisfeiler-Lehman graph isomorphism test, which evaluates whether two graphs are topologically identical. A Graph Neural Network that represents each node by recursively aggregating the neighboring representation. The node representation is updated at each layer k , by aggregating the representations of its neighbors via a sum aggregator, followed by an MLP:

$$h_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right)$$

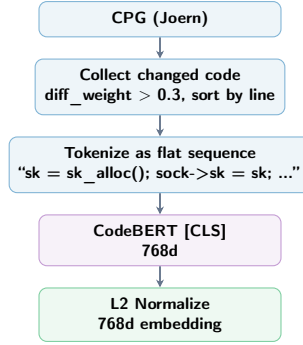
Where ϵ is either a learnable paramet or a fixed scalar (TODO: check what is the case here). $h_v^{(0)}$ is initialized from the node features. (TODO: explain network initialization). The number of layers is equivalent to the number of neighbor hops that will be encoded by each node. This choice is a trade-off of the amount of local structural pattern against the global one. Therefore it learns to embed the subtree in a low dimensional space, not only discriminating different structures but also mapping similar embeddings for similar structures and capturing dependencies between graph structures. Graph-level embeddings are obtained via dual pooling concatenating sum pooling and mean pooling outputs. The sum pooling preserves structural size information, while mean pooling makes the normalized representations normalized.

$$\mathbf{h}_G = \left[\sum_v h_v^{(K)} \parallel \frac{1}{|V|} \sum_v h_v^{(K)} \right]$$

- **Combined.** Concatenates NetLSD, WL and GIN descriptors ($3 \times 128 = 384$ dimensions) and reduces via PCA to the target dimension, by default 128. It aims to be most robust to individual embedder failures.

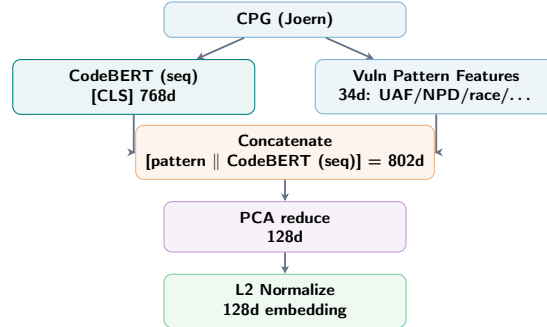


CodeBERT (seq)



Text-only: structure is **discarded**.

CodeBERT (pat)



Fuses **graph structure** with semantics.

PCA and L2 normalization Discussion with experiments
FAISS and HNSW

Agent Architecture

the agent may never call the `submit_patch`, what might result in a parser error in the evaluator, since there is not patch snippet to be evaluated.

The pipeline was implemented in layers that are cleanly separated. In a larger scale, we can split the pipeline in retrieval and generation, where the retrieval happens independently and before the agent ever runs. The agent's receives the retrieved context + vulnerable code and is responsible to return a `*patch*`.

Just like the retrieval, the generation code is built on top of abstractions that allow easier extension of different implementation for experimentation purposes.

We have three dataclasses through which the flow passes:

- `PatchContext` — the input packet: `example_db` (the retrieved similar CVE's meta-data), `target_db` (the CVE to fix), `target_code` (vulnerable C source), and optionally `graph_context` + `query_pair` (CPG graphs for graph-aware variants).
- `AgentResult` — the output packet: `'parsed'` (the `'PatchResult'` with `'cot'` + `vuln_patch`), `'turns'` (one `'InvocationRecord'` per LLM call), `tool_calls` (log of tool executions), `prompt_fingerprint`, and token totals.
- `Agent (Protocol)` — anything with `'name'` and `'run(ctx: PatchContext) -> AgentResult'`.

To allow the easy switch between different agents architecture, there is a `registry.py`. An architecture is identified by name and selectable via `'--architecture'` on the CLI or `config.yaml agents.architecture`:

The goal is to keep the architecture as simple as possible, so we are starting with a simple agent called `single_turn` (located `single_turn.py`) It consists of one LLM call, few-shot prompt and, a marker-parsed output.

Structure-aware tooling. ReAct Yao, Zhao, Yu, et al., 2023 loop where the LLM reasons and calls tools iteratively until it reaches a submission point and it calls `submit_patch`. It may also happen that the agent hits the maximum number of iterations, which is set to 6.

The pipeline has the following functions available as tools:

Backend abstraction. The pipeline has a LLM Transport (`srcagentsbackends.py`) abstraction that is defined by a protocol (`CompletionBackend`) that allows different LLM completion. With this we can run part of the pipeline without rerun the whole

Figure 4.1: Main methods of the patching pipeline with agent

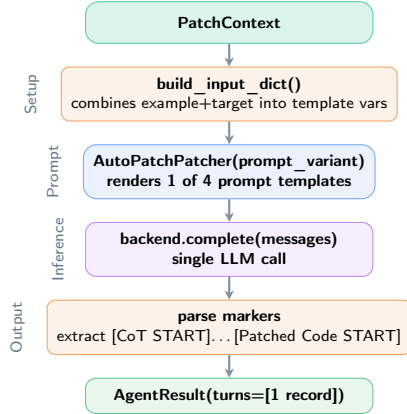
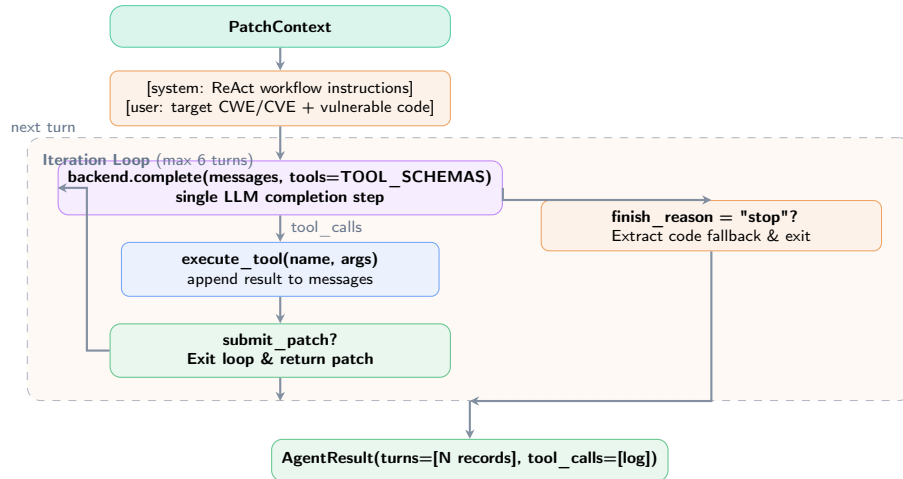


Figure 4.2: Workflow steps of tool calling agent



pipeline. Also, it allows record and mocked responses so we can experiment only part of the pipeline without incurring high costs. The `complete()` signature was extended with `tools=` and `tool_choice=` kwargs, so the same backend handles both architectures. Four implementations:

To swap between backends, it is only necessary to call `use_backend()` context manager, no other call change is needed.

Agent roles. Structure-aware prompt construction.

Patching architectures (experimental axis).

Table 4.1: List of the tools that the agent can call throughout its pipeline.

Tool	Functionality
<code>get_vulnerability_context</code>	Returns CVE description, CWE category, and typical fix patterns from the static knowledge base (<code>cwe_descriptions.py</code>). It covers CWE-20/190/362/400/416/476/787
<code>analyze_example_fix</code>	Calculate the difference between <code>example_db.original_code</code> and <code>example_db.vuln_patch</code> and returns a structured unified diff (truncated to a token budget). Also return a record of the fixes (<code>fix_list</code>)
<code>check_c_syntax</code>	Parses the patch with <code>tree-sitter-c</code> if available, falls back to brace-balance heuristics
<code>verify_fix_correctness</code>	Calls a second LLM (same deployment by default) as a judge to check whether the patch addresses the root cause
<code>submit_patch</code>	Records the final patch + reasoning and exits the loop

Versioned prompts.

Reproducibility and cost tracking.

Evaluation

Also included LLM-as-a-judge evaluator inspired on Shi, Wei, Tufano, et al., 2026

Backend	Function
LiveBackend	Azure/litellm call (lazy import)
MockBackend	Deterministic output for unit tests
RecordBackend	Wraps Live, records to JSONL cassette
ReplayBackend	Reads from cassette, zero network

5 Results

Per-record fingerprints:

`single_turn` → 34b727c79607 (the few-shot marker prompt from the registry, via `single_turn.py`)

`tool_calling` → “ empty — it uses its own hardcoded ReAct `_SYSTEM_PROMPT` + tool schemas in `src/agents/architectures/tool_calling.py`

The `run_meta.json` showed one fingerprint for both only because that field is computed at the run level from the "default" variant name, regardless of what the architecture actually does internally. So they differ in both prompt and control-flow.

What’s actually shared: the model (gpt-4o), the retrieved example, the target code, and the split. And the real reason the surface metrics look similar isn’t the prompt — it’s that both outputs are dominated by the unchanged function body they copy verbatim, which saturates BLEUROUGEJaccard. So the correct statement is: "different prompts and control flow, but surface metrics can’t see the difference because copied boilerplate dominates them."

How to evaluate adding a multi-agent architecture

What direction actually gets better results

multi-agent is worth measuring, but given the 29

5.0.1 File vs. Method

After investigation, I found that the method graphs were actually degenerated, that is, Joern was not able to understand the code elements resulting in a graph of unique node element, containing the whole code and edges have as source the node but the other end is null. This behavior could result from the lack of the definition of all elements definition, making it difficult for the joern create the graphs.

Because the dataset also contain the file, we’ve done a comparison of the performance using the file and the method as input.

The results are in the Table 5.1. They show that file-level graphs outperform the method-level graphs, especially for the combined embedder for which `hit@1` jumps from 0.333 to 0.889 (+0.556), and `mrr` jumps from 0.454 to 0.944 (+0.491). `CodeBERT_pattern` `hit@1` increases from 0.678 to 0.889 (+0.211). `CodeBERT_seq` has a more modest `hit@1` improvement from 0.722 to 0.889 (+0.167). These improvements shows that the extra

Table 5.1: Comparison of CVE Retrieval Performance using file and method as input

Table 5.2: Performance Comparison across Embedders

Embedder	Metric	Method	File	Delta	Direction
codebert_pattern	hit@1	0.678	0.889	+0.211	↑
	hit@5	0.789	0.889	+0.100	↑
	hit@10	0.789	0.889	+0.100	↑
	mrr	0.720	0.889	+0.169	↑
	cwe_recall	0.619	0.650	+0.031	↑
codebert_seq	hit@1	0.722	0.889	+0.167	↑
	hit@5	0.778	0.889	+0.111	↑
	hit@10	0.789	0.889	+0.100	↑
	mrr	0.741	0.889	+0.148	↑
	cwe_recall	0.627	0.605	-0.022	↓
combined	hit@1	0.333	0.889	+0.556	↑
	hit@5	0.656	1.000	+0.344	↑
	hit@10	0.811	1.000	+0.189	↑
	mrr	0.454	0.944	+0.491	↑
	cwe_recall	0.384	0.750	+0.366	↑

content encoded in the file-level graph bring important context which may include global variables, imports, and cross-method, resulting in structures that leads to a better vulnerability identification. This improvements make that the final performance of the embedders are equivalent. This allows us to conclude that enough context information allows a simple embedder of graph topographic description to perform equivalently to a semantic level pretrained model.

Table 5.3: Patch Evaluation Metrics Across CWEs GPT-5.3-Codex, CodeBERT Sequence Embedder

Category	Count	Exact Match	Hunk F1	BERTScore F1	Char Ratio	Norm. Edit Dist.	BLEU-4
CWE-190	15	5	0.6149	0.6960	0.7735	–	0.7504
CWE-20	15	3	0.8643	0.9121	0.8581	–	0.8579
CWE-362	15	8	0.9983	0.9648	0.9966	–	0.9912
CWE-416	14	10	0.9344	0.9387	0.9706	–	0.9593
CWE-476	14	9	0.9007	0.9334	0.9001	–	0.8910
CWE-787	15	0	0.2625	0.5243	0.8732	–	0.8941
General (All)	88	35	0.7590	0.8258	0.8944	0.1051	0.8899

One example did not complete because it reached the maximum of rounds, six.

List of Figures

4.1	Patch pipeline	17
4.2	Tool Agent Pipeline	17

List of Tables

3.1	Distribution of CWE Types in the AutoPatch Dataset ($N = 75$)	6
3.2	CWE Frequency Distribution Across Index and Query (Test) Datasets .	8
3.3	Frequency of Top CVEs Across Index, Query, and Combined Datasets .	9
4.1	Available Tools Table	18
5.1	Comparison of CVE Retrieval Performance using file and method as input	21
5.2	Performance Comparison across Embedders	21
5.3	Patch Evaluation Metrics Across CWEs GPT-5.3-Codex, CodeBERT Sequence Embedder	22

Bibliography

- Du, X., Zheng, G., Wang, K., Zou, Y., Wang, Y., Deng, W., Feng, J., Liu, M., Chen, B., Peng, X., Ma, T., & Lou, Y. (2025). Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag.
- Guo, C., Nie, Y., Xie, C., Lin, Z., Guo, W., & Li, B. (2025). Bluecodeagent: A blue teaming agent enabled by automated red teaming for codegen ai.
- International Organization for Standardization. (2022). *Iso/iec 27001:2022 information security, cybersecurity and privacy protection — information security management systems — requirements* (Standard). International Organization for Standardization. Geneva, Switzerland.
- Seo, M., Choi, W., Shin, S., & You, M. (n.d.). Autopatch: Multi-agent framework for patching real-world cves generated by outdated llms. *Available at SSRN 6339960*.
- Shi, S., Wei, R., Tufano, M., Cambronero, J., Cheng, R., Ivancic, F., & Rondon, P. (2026). Towards a human-in-the-loop framework for reliable patch evaluation using an llm-as-a-judge. *Proceedings of the 1st International Workshop on AI for Software Quality Evaluation - Judgment, Metrics, Benchmarks, and Beyond*, 19–28. <https://doi.org/10.1145/3786175.3788348>
- Tsitsulin, A., Mottin, D., Karras, P., Bronstein, A., & Müller, E. (2018). Netlsd: Hearing the shape of a graph. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2347–2356. <https://doi.org/10.1145/3219819.3219991>
- Wang, L., Jiang, J., et al. (2025). Valign-glar: Graph retrieval-based vulnerability intelligence alignment via structured llm-guided inference. *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS '25)*. <https://doi.org/10.1145/3733800.3763266>
- Xia, C. S., & Zhang, L. (2024). Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 819–831. <https://doi.org/10.1145/3650212.3680323>
- Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? *International Conference on Learning Representations (ICLR)*.

- Yang, Y., Xu, B., Gao, X., & Sun, H. (2025). Context-enhanced vulnerability detection based on large language models. *ACM Trans. Softw. Eng. Methodol.* <https://doi.org/10.1145/3779222>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. React: Synergizing reasoning and acting in language models [Publisher Copyright: © 2023 11th International Conference on Learning Representations, ICLR 2023. All rights reserved.; 11th International Conference on Learning Representations, ICLR 2023 ; Conference date: 01-05-2023 Through 05-05-2023]. English (US). In: Publisher Copyright: © 2023 11th International Conference on Learning Representations, ICLR 2023. All rights reserved.; 11th International Conference on Learning Representations, ICLR 2023 ; Conference date: 01-05-2023 Through 05-05-2023. 2023.